Repaso Lección #1

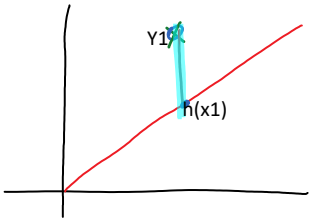
miércoles, 25 de octubre de 2023 20:27

Aprendizaje Supervisado - Regresión Lineal de una variable:

Predecir una variable de respuesta dependiente en función de una variable predictora (independiente)

$y = w_0 + w_1 x$

$$h_w(x) = w_0 + w_1 x \ \ (hipótesis\ -\ predicción)$$



Error Cuadrático medio MSE

y: Valor real (Data que se conoce)
h(x): Valor que se obtiene, predicción del modelo.

Error: $(h_w(x^i) - y^i)^2$ -> Es la diferencia en cada punto

Función de costo

Suma de todas las diferencias = $\frac{1}{2m}\sum_{i=1}^{m}(h_w(x^i) - y^i)$; m = total de puntos

Error total promedio = $J(w_0, w_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_w(x^i) - y^i)^2$

Poco a poco ver con cada valor de w_0, w_1, cuál tiene menos error.

Minimizar función de costo

- Tratar que $J(w_0, w_1)$ sea igual a 0

Procedimiento:

- Valores iniciales de w0 y w1
- Actualizar w0 y w1 hasta reducir $J(w_0, w_1)$ -> Reducir el error total promedio
- Hasta que el error se acerque a 0 -> llegar mínimo global.

Para actualizar w0 y w1 -> Gradiente descendiente -> Regla de aprendizaje

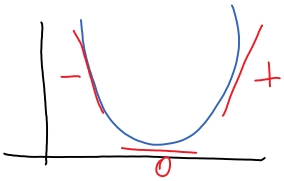
Gradiente -> Dirección y magnitud de cambios.

- Derivada de la función $J(w_0, w_1)$ en función de w_0 y w_1

$$w_{0\ nuevo} \coloneqq w_{0\ anterior} - \propto \frac{d}{dw_0} J(w_0)$$

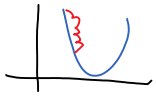
$\frac{d}{dw_0} J(w_0)$ -> Gradiente -> Derivada parcial con respecto a w_0

- La gradiante es positiva si la pendiente en ese punto de la función es creciente
- La gradiente es negativa si la pendiente en ese punto de la función es decreciente
- La gradiente es 0 si está en un mínimo local

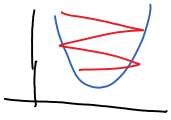


$\propto \rightarrow Velocidad\ de\ aprendizaje \rightarrow Hiperparámetro$

- Si $\propto$ es pequeño, la gradiente es lenta.



- Si $\propto$ es grande, la gradiente puede sobrepasar el mínimo. Converge o diverge -> over-shooting



Finalmente, repetir hasta que llegue la mínimo:

$$w_{0\ nuevo} \coloneqq w_{0\ anterior} - \propto \frac{d}{dw_0} J(w_0)$$

Nota: Cálculo de derivadas parciales:

$\frac{d}{dw_0} J(w_0, w_1)$ -> Tratar w1 como una constante

Regla de la potencia:

La derivada de x^n es nx^{n-1}

$$\frac{d}{dx}(x^n) = nx^{n-1}$$

Pasos:

1) Iniciar con valores aleatorios w1,w2…
2) Calcular la salida hw(x) (Valor calculado)
3) Comparar con y (Valor esperado)
4) Calcular J (Error total)
5) Obtener derivada de J con respecto a w1, w2, etc (Gradiente)
6) Se corrige el error con la fórmula wnueva = wantigua - alpha (gradiente)

Aprendizaje Supervisado - Regresión Lineal de Múltiples Variables:

Predecir una variable de respuesta dependiente en función de varias variables predictoras

Variable1 x^1	Variable2 x^2	Variable3 x^3	Variable4 x^4	Valor esperado y
210	5	2	45	460
141	3	1	40	232
${x_1}^3$	${x_2}^3$	${x_3}^3$	${x_4}^3$	y^3
…	…	…	…	…

M

N

M -> cantidad de ejemplos
N -> cantidad de variables - features
x^i -> Variables de entrada del i ejemplo -> x^1 = [210,5,2,45]
${x_j}^i$ -> Valor de variable j en el i ejemplo -> ${x_4}^1$ = 45

Para actualizar todas las w's -> Gradiente descendiente -> Regla de aprendizaje

$$h_w(x) = w^T X = w_0x_0 + w_1x_1 + w_2x_2 \dots + w_nx_n \ (hipótesis - predicción)$$
$$w_0, w_1, w_2 \dots , w_n \ (Parametros)$$
$$J(w_0, w_1, w_2 \dots w_n) = \frac{1}{2m}\sum_{i=1}^{m}(h_w(x^i) - y^i)^2 \ (Función\ de\ costo)$$

Algoritmo Gradiente Descendiente:

$$w_{jnuevo} \coloneqq w_{j\ anterior} - \propto \frac{1}{m}\sum_{i=1}^{m}(h_w(x^i) - y^i) * {x_j}^i$$

$${x_j}^i = es\ el\ valor\ de\ la\ característica\ j\ para\ el\ i'esimo\ ejemplo$$

Aprendizaje Supervisado - Regresión polinómica:

En lugar de ajustar una línea recta a los datos, la regresión polinómica ajusta una curva polinómica a los datos.

$$h_w(x) = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nw_n$$
$$h_w(x) = w_0 + w_1(tamaño)^2 + w_2(tamaño)^3 \dashrightarrow Cubica$$

Ejemplos:

$$h_w(x) = w_0 + w_1{x_1}^2 \dashrightarrow Cuadratica$$
$$h_w(x) = w_0 + w_1{x_1}^2 + w_2{x_2}^3 \dashrightarrow Cubica$$

UnderFitting - Overfitting

Underfitting: Muy simple - grado muy pequeño
Overfitting: Muy Complejo - memorización - no permite generalizar - grado muy grande

Regularización:

Evitar el sobreajuste. Agrega un término adicional a la función de costo. Penaliza la complejidad del modelo. Previene sobreajuste, controla complejidad.

Overfitting:

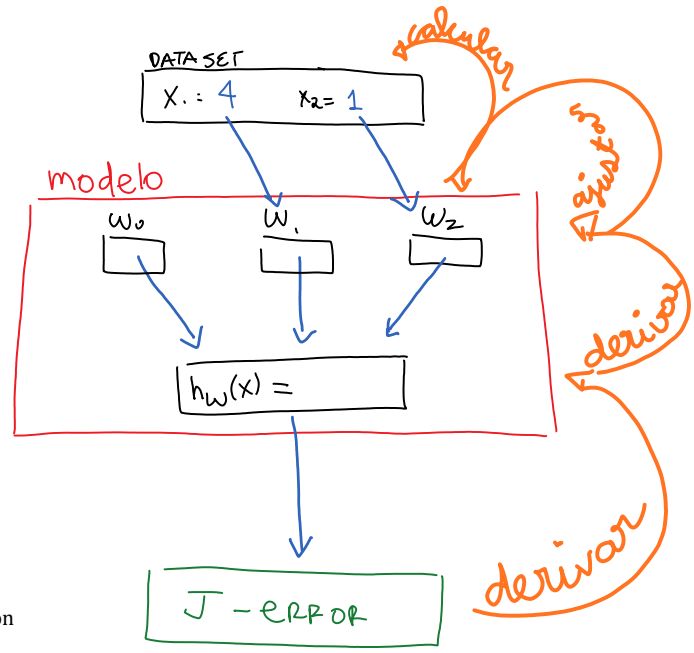
Regularización L1 (Lasso)	Agrega término de penalización. Proporcional al valor absoluto coeficientes. **Intenta que coeficientes sean 0**. Selección de características.	Modelo: $10000x_1 + 5000x_2 + 0x_3$ Se forzó a que x3 sea 0.
Regularización L2 (Ridge)	Agrega término de penalización proporcional al cuadrado de coeficientes. Penaliza valores extremadamente grandes, **reduce magnitud, no vuelve 0.**	$Modelo: 9000x_1 + 4500x_2 + 300x_3$ Se reducen coeficientes pero no coloca 0

Propagación hacia adelante

Se tiene:

$h_w(x) = w_0 + w_1x_1 + w_2x_2$
$w_0 = 0\ ;\ w_1 = -0.017;\ w_2 = -0.048$

	X1	X2	y
1	4	1	2



1) Calcular con el modelo

$h_w(x) = w_0 + w_1x_1 + w_2x_2$
$h_w(x) = 0 + (-0.017)\ 4 + (-0.048)1$
$h_w(x) = -0.116$

2) Calculamos el error

Y = 2 (Valor real)
J = (h_w(x) - y)2 = (-0.116 - 2)2 = 4.48

Si y es muy diferente a J, pasar al paso 3

3) Calcular las derivadas - Saber el punto de dirección

$h_w(x) = w_0 + w_1x_1 + w_2x_2$

3.1)Calcular derivada de h(x) con respecto a J

$J = (hw(x) - y)^2$ -> Original
$\frac{\partial j}{\partial hw(x)} = 2(hw(x) - y\ )$ -> Derivada que se usará a continuación

3.2 Calcular derivada w0 con respecto a H(x)

$$\frac{\partial hw(x)}{\partial w0} = 1 + w_1 0 + w_2 0$$

$$\frac{\partial hw(x)}{\partial w0} = 1$$

3.3 Calcular derivada w1 con respecto a H(x)

$$\frac{\partial hw(x)}{\partial w1} = 0 + x1 + 0$$

$$\frac{\partial hw(x)}{\partial w1} = x1$$

3.4 Calcular derivada w2 con respecto a H(x)

$$\frac{\partial hw(x)}{\partial w2} = 0 + 0 + x2$$

$$\frac{\partial hw(x)}{\partial w2} = x2$$

4) Ajustar valores de w0, w1, w2

$$w_{j\ nuevo} \coloneqq w_{j\ anterior} - \propto \frac{d}{dw_j} J\left(w_j\right)$$

Y = 2; $\propto$ = 0.05
Hw(x) = -0.116

4.1) Ajustar w0

$w_{0\ nuevo} \coloneqq w_{0\ anterior} - \propto [(Derivada\ hw(x) con\ J\ - y\ ) * Derivada\ de\ w0\ con\ h(x)]$
$w_{0\ nuevo} \coloneqq 0\ - 0.05\ * [2(hw(x) - y) * 1]$
$w_{0\ nuevo} \coloneqq 0\ - 0.05\ * [2(-0.116\ - 2\ )]$
$w_{0\ nuevo} \coloneqq\ -0.05\ * [2(-2.116)]$
$w_{0\ nuevo} \coloneqq 0.212$

4.1) Ajustar w1

X1 = 4
$w_{1\ nuevo} \coloneqq w_{1\ anterior} - \propto [(Derivada\ hw(x) con\ J\ - y\ ) * Derivada\ de\ w0\ con\ h(x)]$
$w_{1\ nuevo} \coloneqq 0\ - 0.05\ * [2(hw(x) - y) * x1]$
$w_{1\ nuevo} \coloneqq 0\ - 0.05\ * [2(-0.116\ - 2\ )\ * 4]$
$w_{1\ nuevo} \coloneqq -0.017\ - 0.05\ * [-16.928]$
$w_{1\ nuevo} \coloneqq 0.829$

4.1) Ajustar w2

X2 = 1

$w_{2\,nuevo} \coloneqq w_{2\,anterior} - \propto [(Derivada\ hw(x) con\ J\ -y\ ) * Derivada\ de\ w0\ con\ h(x)]$
$w_{2\,nuevo} \coloneqq 0\ -0.05\ * [2(hw(x) - y) * x2]$
$w_{2\,nuevo} \coloneqq 0\ -0.05\ * [2(-0.116\ -2\ ) * 1]$
$w_{2\,nuevo} \coloneqq -0.048\ -0.05\ * [-4.232]$
$w_{2\,nuevo} \coloneqq 0.1636$

Formas de clasificación de Batches:

Batch -> Usa el error promedio de TODO el conjunto de datos.
- Calcula w0,w1 y w2 para cada fila en la tabla.
- Calcula sus errores respectivos
- Promedia.
- Modifica w0, w1 y w2

Mini-batch -> Separa las filas en pequeños grupos. Y calcula por cada grupo como si fuera batch.
- Luego cada mini-batch siguiente se usa el cálculo del minibatch anterior

Regresión Logística (Clasificación)

Predecir probabilidad de un ejemplo pertenezca a una de las clases de un problema de clasificación binaria.
Se habla de probabilidades: h(x) probabilidad que x sea 1 o 0. Si h(x) = 0.7 entonces es 70% de probabillidades.

Si $h_w(x) \geq\ \emptyset\ , predecir\ y = 1$
Si $h_w(x) <\ \emptyset\ , predecir\ y = 0$
$\emptyset = 0.5$

Queremos que sean valores entre [0,1] por lo tanto:
$h_w(x) = \frac{1}{1+ e^{-w^T x}}$ -→ *hace que si o si el valor sea entre 0 y 1*
$Z = w^T x$

$$h_w(x) = \frac{1}{1+ e^{-z}}$$

Función de costo

Mide el error

Costo(hw(x),y) = $-ylog(hx(x)) - (1\ -y) \log(1\ -hw(x))\ \ (Forma\ extendida)$

Si y = 1: Costo(hw(x),y) = $-ylog(hx(x))$
Si y = 0: Costo(hw(x),y) = $-\log(1\ -hw(x))$

Gradiente descendiente:

J(W) = $-\frac{1}{m}\sum_{i=1}^{m} ylog(hx(x)) - (1\ -y) \log(1\ -hw(x))$

Regla de aprendizaje

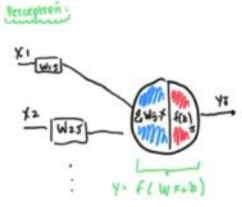
$$w_{jnuevo} \coloneqq w_{j\ anterior} - \propto \frac{1}{m}\sum_{i=1}^{m}(h_w(x^i) - y^i)^{\square} * {x_j}^i$$

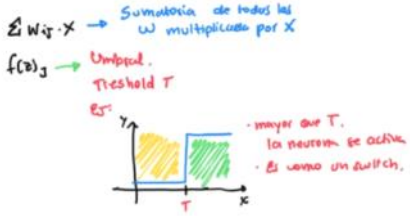
Recordando que hw(x) es:

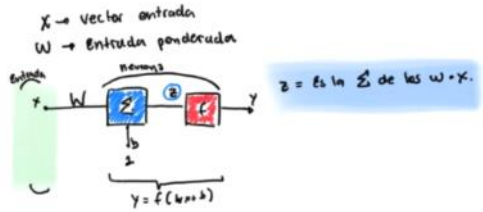
$$h_w(x) = \frac{1}{1+ e^{-z}}$$

Redes Neuronales:

Perceptrón:

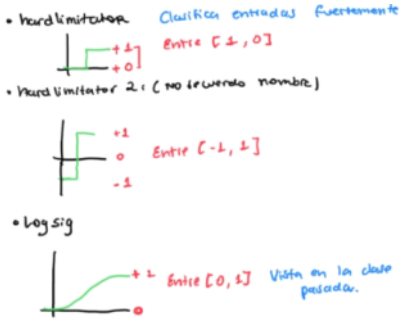






- El Threshold es un umbral. Es lo que dice que se debe seguir modificando W's. Puede ser + o -

Funciones de activación:



Hardlimitator: Funcion de transferencia hard
Logsig: Funci'on de transferencia soft

Función de transferencia "hard"

- El umbral se activa cuando:
  - Suma de entradas * pesos
  - BIAS
  - Ambos positivos

Función de transferencia "soft"

- Suaviza la salida.
- El umbral se activa cuando:
  - Función continúa en la combinación ponderada de entradas.

Ejemplo Perceptrón:

X1	X2	Z	y
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

Y = 0 -> Cuando n no cumple Threshold
Y = 1 -> Cuando n si cumple Threshold

1) Modelo: $x_1 w_1 + x_2 w_2 \geq 1.5$
2) Función de activación hard. 0 y 1
3) Operaciones:
   a. X1 = 0, X2 = 0

      $z = 0 \geq 1.5\ (Falso)$

   b. X1 = 0, x2 = 1

      $z = 1 \geq 1.5\ (Falso)$

   c. X1 = 1, x2 = 0

      $z = 1 \geq 1.5\ (Falso)$

   d. X1 = 1, x2 = 1

      $z = 2 \geq 1.5\ (Verdadero)$

Redes neuronales:

Profundidad: Longitud del camino más largo

- Son funciones universales de aproximación
- Modelar cualquier función Boolean
- Modelar cualquier border de clasificación
- Modelar cualquier función de valores continuos

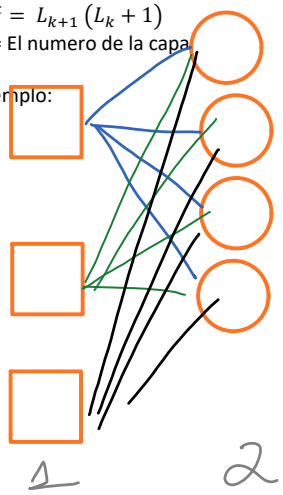
Peso:

Intensidad de la conexión entre neuronas. Más peso, más intensidad, más importancia a la neurona. Formato Matriarcial.

Obtener el tamaño de la matriz:
Una capa L tiene N neuronas, y la siguiente L + 1 tiene M neuronas -> Matriz NXM

$w^k = L_{k+1}\,(L_k + 1)$
K = El numero de la capa

Ejemplo:



$w^k = L_{k+1}\,(L_k + 1)$
$w^1 = L_{1+1}\,(L_1 + 1)$

$L_1 = 3\ \ (Hay\ 3\ neuronas\ en\ la\ capa\ 1)$
$L_2 = 4\ \ (Hay\ 4\ neuronas\ en\ la\ capa\ 1)$

$w^1 = 4\,(3 + 1)$
$w^1 = 4\ x4\ (Tamaño\ de\ la\ matriz)$

Tipos de Pesos:

Pesos Fijos:
- Constantes que no cambian durante el entrenamiento
- Cuando se requiere una parte específica de la red - se conoce de antemano el valor óptimo

Pesos Adaptables:
- Valores que se ajustan automáticamente durante el entrenamiento
- Valores aleatorios
- Red puede aprender y adaptarse.
- Función que buscamos en la red Neuronal

Multi Layer Perceptron:

Una capa -> Hemiplano limitado por hiperplano
Dos capas -> Regiones convexas abiertas o cerradas
Tres capas -> Arbitrarias (Complejidad limitada por N de nodos)

${x_{ij}}^k$ -> k: Numero de capa. Indice i de donde viene . Indice J, hacia donde se dirige

Teniendo:

$a^1 = f^1(W^1x + b^1) \rightarrow Ecuación\ para\ la\ primer\ capa.\ La\ entrada\ es\ x\ y\ la\ salida\ es\ a^1$

$a^2 = f^2(W^2a^1 + b^2) \rightarrow Ecuación\ para\ la\ segunda\ capa.\ La\ entrada\ es\ a^1\ y\ la\ salida\ es\ a^2$

Tipos de aprendizajes:

Supervisado
- Proporciona ejemplos-etiquetas (Entrada y salida) (x, y)
- Evalúa comparando con etiquetas reales
- La red ajusta sus pesos para minimizar el error
- Clasificación y regresión
- Ej: Reconocimiento de patrones

No Supervisado
- No se proporcionan etiquetas (x,y) sólo datos de entrada x
- Modelo descubre patrones
- Clustering (Agrupamiento)

Con refuerzo
- Proximamente

Transferido
- Proximamente

Entrenamiento:

Para cada peso W se realiza un ajuste, regla delta:
$\Delta W_{ij} = \alpha(T_j - Y_j)Y_i$

$\alpha = Velocidad\ de\ aprendizaje\ entre\ 0\ y\ 1$
$T_j = Valor\ deseado$
$Y_j = Valor\ obtenido$
$Y_i = Valor\ de\ salida\ de\ la\ neurona\ i$

Pasos:

1) Seleccionar α
2) Inicializar pesos w con valores aleatorios
3) Clasificar todos los ejemplos correctamente
    a. Ingresar nuevo ejemplo
        i. Si ejemplo incorrecto:
        ii. $W_{nuevo} = W_{anterior} - \propto (t\ - y\ )x$